

2.4 - An Anagram Detection Example

A good example problem for showing algorithms with different orders of magnitude is the classic anagram detection problem for strings. One string is an ***anagram*** of another if the second is simply a rearrangement of the first. For example, heart and earth are anagrams. The strings taster and treats are anagrams as well. For the sake of simplicity, we will assume that the two strings in question are of equal length and that they are made up of symbols from the set of 26 lowercase alphabetic characters. Our goal is to write a boolean function that will take two strings and return whether they are anagrams.

2.4.1 - Solution 1: Anagram Detection Checking Off

Our first solution to the anagram problem will check the lengths of the strings and then check to see that each character in the first string actually occurs in the second. If it is possible to check off each character, then the two strings must be anagrams. ***Checking off*** a character will be accomplished by setting an element to - if it is found (it's not a letter, so it won't ever match letters in the first string).

However, since strings in Java are immutable, the first step in the process will be to convert the second string to an array of characters with the data type `char`. The `char` data type stores exactly one character, and we can use `==` to compare them directly (unlike `Strings`).

We will get each character from the first string by using the `charAt()` method and check it against the characters in the array formed from the second string. If a letter is found, we check it off by setting the array element to - . [Listing 2.4.1](#) shows this method.

Listing 2.4.1.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 public class Anagrams1 {
2
3     public static boolean anagramSolution1(String s1, String
4         boolean isAnagram = true;
5
6     if (s1.length() != s2.length()) {
7         isAnagram = false;
8     } else {
9         int pos1 = 0;
10        char[] s2Array = s2.toCharArray();
11        while (pos1 < s1.length() && isAnagram) {
12            int pos2 = 0;
13            boolean found = false;
14            while (pos2 < s2.length() && !found) {
15                if (s1.charAt(pos1) == s2Array[pos2]) {
16                    found = true;
17                } else {
18                    pos2 = pos2 + 1;
19                }
20            }
21            if (found) {
22                s2Array[pos2] = '-';
23            } else {
24                isAnagram = false;
25            }
26            pos1 = pos1 + 1;
27        }
28    }
29    return isAnagram;
30 }
```

To analyze this algorithm, we need to note that each of the n characters in s_1 will cause an iteration through up to n characters in the list from s_2 . Each of the n positions in the s_2 array will be visited once to match a character from s_1 . The number of visits then becomes the sum of the integers from 1 to n . We stated earlier that this can be written as

$$\begin{aligned}\sum_{i=1}^n i &= \frac{n(n+1)}{2} \\ &= \frac{1}{2}n^2 + \frac{1}{2}n\end{aligned}$$

As n gets large, the n^2 term will dominate the n term and the $\frac{1}{2}$ can be ignored. Therefore, this solution is $O(n^2)$.

This is a worst-case analysis, and it happens when the strings are anagrams—the algorithm has to compare every pair of characters. If the words are not anagrams, the algorithm will exit before all the comparisons are done (as soon as a mismatch is found).

2.4.2 - Anagram Detection Solution 2: Sort and Compare

Another solution to the anagram problem will make use of the fact that even though s_1 and s_2 are different, they are anagrams only if they consist of exactly the same characters. So if we begin by sorting each string alphabetically from a to z, we will end up with the same string if the original two strings are anagrams. [Listing 2.4.2](#) shows this solution. Again, in Java we can use a `sort` method on arrays by converting each string to an array at the start.

Listing 2.4.2.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 import java.util.Arrays; // note the "s" at the end!
2
3 public class Anagrams2 {
4
5     public static boolean anagramSolution2(String s1, String
6         char[] s1Array = s1.toCharArray();
7         char[] s2Array = s2.toCharArray();
8
9         Arrays.sort(s1Array);
10        Arrays.sort(s2Array);
11
12        int pos = 0;
13        boolean matches = true;
14
15        while (pos < s1.length() && matches) {
16            if (s1Array[pos] == s2Array[pos]) {
17                pos = pos + 1;
18            } else {
19                matches = false;
20            }
21        }
22        return matches;
23    }
24
25    public static void main(String[] args) {
26        System.out.println(anagramSolution2("taster", "treat");
27        System.out.println(anagramSolution2("abcd", "dcab"));
28        System.out.println(anagramSolution2("abcd", "dcda"));
29    }
30 }
```

At first glance you may be tempted to think that this algorithm is $O(n)$, since there is one simple iteration to compare the n characters after the sorting process. However, the two calls to the Java `sort` method are not without their own cost. As we will see in Chapter 5, sorting is typically either $O(n^2)$ or $O(n \log n)$, so the sorting operations dominate the iteration. In the end, this algorithm will have the same order of magnitude as that of the sorting process.

2.4.3 - Anagram Detection Solution 3: Brute Force

A **brute force** technique for solving a problem typically tries to exhaust all possibilities. For the anagram detection problem, we can simply generate a list of all possible strings using the characters from `s1` and then see if `s2` occurs. However, there is a problem with this approach. When generating all possible strings from `s1`, there are n possible first characters, $n - 1$ possible characters for the second position, $n - 2$ for the third, and so on. The total number of candidate strings is $n \cdot (n - 1) \cdot (n - 2) \dots \cdot 3 \cdot 2 \cdot 1$, which is $n!$. Although some of the strings may be duplicates, the program cannot know this ahead of time and so it will still generate $n!$ different strings.

It turns out that $n!$ grows even faster than 2^n as n gets large. In fact, if `s1` were 20 characters long, there would be $20! = 2,432,902,008,176,640,000$ possible candidate strings. If we processed one possibility every second, it would still take us 77,146,816,596 years to go through the entire list. This is probably not going to be a good solution.

2.4.4 - Anagram Detection Solution 4: Count and Compare

Our final solution to the anagram problem takes advantage of the fact that any two anagrams will have the same number of a's, the same number of b's, the same number of c's, and so on. In order to decide whether two strings are anagrams, we will first count the number of times each character occurs. Since there are 26 possible characters, we can use an array of 26 counters, one for each possible character. Each time we see a particular character, we will increment the counter at that position. In the end, if the two lists of counters are identical, the

strings must be anagrams. We will take advantage of the fact that char variables in Java are stored as integers, and Java allows us to evaluate an expression like `'c' - 'a'`, which evaluates to 2. [Listing 2.4.3](#) shows this solution.

Listing 2.4.3.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 import java.util.Arrays;
2
3 public class Anagrams4 {
4
5     public static boolean anagramSolution4(String s1, String
6         int[] count1 = new int[26]; // initialized to all zero
7         int[] count2 = new int[26];
8
9         for (int i = 0; i < s1.length(); i++) {
10             int index = s1.charAt(i) - 'a';
11             count1[index] = count1[index] + 1;
12         }
13
14         for (int i = 0; i < s2.length(); i++) {
15             int index = s2.charAt(i) - 'a';
16             count2[index] = count2[index] + 1;
17         }
18
19         int j = 0;
20         boolean isAnagram = true;
21         while (j < 26 && isAnagram) {
22             if (count1[j] == count2[j]) {
23                 j = j + 1;
24             } else {
25                 isAnagram = false;
26             }
27         }
28         return isAnagram;
29     }
30 }
```

Again, the solution has a number of iterations. However, unlike the first solution, none of them are nested. The first two iterations used to count the characters are both based on n . The third iteration, comparing the two lists of counts, always takes 26 steps since there are 26 possible characters in the strings. Adding it all up gives us $T(n) = 2n + 26$ steps. That is $O(n)$. We have found a linear order of magnitude algorithm for solving this problem.

Before leaving this example, we need to say something about space requirements. Although the last solution was able to run in linear time, it could only do so by using additional storage to keep the two lists of character counts. In other words, this algorithm sacrificed space in order to gain time.

This is a common occurrence. On many occasions you will need to make decisions between time and space trade-offs. In this case, the amount of extra space is not significant. However, if the underlying alphabet had millions of characters, there would be more concern. As a computer scientist, when given a choice of algorithms, it will be up to you to determine the best use of computing resources given a particular problem.



Self Check

1.

Q-4: Given the following code fragment, what is its Big O running time?
(Presume that variable *n* has been declared elsewhere in the program.)

```
int test = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        test = test + i * j;
    }
}
```

Choose one

- ☐ A. $O(n)$
- ☒ B. $O(n^2)$
- ☐ C. $O(\log n)$
- ☐ D. $O(n^3)$

Check Me

Compare me

2.

Q-5: Given the following code fragment what is its Big O running time?
(Presume that n has been declared elsewhere in the program.)

```
int test = 0;
for (int i = 0; i < n; i++) {
    test = test + i;
}

for (int j = 0; j < n; j++) {
    test = test - 1;
}
```

Choose one

- ☒ A. $O(n)$
- ☐ B. $O(n^2)$
- ☐ C. $O(\log n)$
- ☐ D. $O(n^3)$

Check Me

Compare me

3.

Q-6: Given the following code fragment what is its Big O running time?
(Presume n has been declared elsewhere in the program.)

```
int i = n;
while (i > 0) {
    int k = 2 + 2;
    i = i / 2;
}
```

Choose one

- ☐ A. $O(n)$
- ☐ B. $O(n^2)$
- ☒ C. $O(\log n)$
- ☐ D. $O(n^3)$

Check Me

Compare me

You have attempted 4 of 7 activities on this page.